

Project Review Criteria

Why this matters. These projects show what people build on GenLayer. When we accept a project, we are saying it is good work, and it represents GenLayer to everyone. Look at each project with care and take your time. Giving a low score is fine. An honest score helps the builder and helps us.

Milestone submissions are not reviewed with this rubric. See the separate section at the end.

How it works

1. Check that the project is valid (the gate).
2. If it is valid, score four categories from 0 to 5.
3. Choose one of three outcomes: **accept**, **request more info**, or **reject**.
4. Send the proposal with a short note for each score and one line of overall opinion.

Three rules

- Look at the real work: the code, the deployed contract, and the live app if there is one.
- Start at 0. Add points only for evidence you can see. Missing evidence counts against the project, not for it.
- Most projects will get a 1 or a 2. A 4 or a 5 is not common, like a unicorn 🦄

Who does what

AI analysis assists with repository inspection, the gate, GenLayer fit, contract quality, and engineering. The human reviewer verifies that analysis, tests the live product, assigns Frontend/UX, reviews optional external evidence, and makes the final decision.

Use AI where the criteria are concrete and checkable. Judgment calls about use case potential, product value, and UI/UX stay with the human reviewer.

Verify, do not trust

Before accepting a project with a live deployment, the human reviewer confirms that:

- the live app loads and is usable;
- wallet and network handling work;
- the main flow reaches the actual contract;
- transaction states and errors are understandable;
- the deployed methods correspond to the repository source;
- the interface is genuinely usable, not only visually polished.

A generated or attractive frontend does not compensate for weak GenLayer work.

Step 1: The validity gate

Reject immediately if any of these is true

- There is no real GenLayer contract, and the project does not qualify as GenLayer-specific tooling (see below).
- The AI part does not run on GenLayer. A centralized backend or API makes the decision and the contract only stores or displays it.
- The project uses the GenLayer name but never calls a contract.
- The code does not run.
- The repository is empty, or it is an unmodified or minimally changed copy of another project or of an official example.
- The frontend claims contract integration but simulates it, using mocked results or server memory.
- The GitHub repository returns 404 or has been deleted.

Reusing a foundation is allowed. Building on an example or a known pattern is fine when substantial new logic, mechanics, or user value is added. Renaming variables is not.

Ecosystem tooling

A project may qualify without defining its own Intelligent Contract when it is GenLayer-specific tooling: developer tools, SDK extensions, explorers, testing infrastructure, integrations. The tooling has to be genuinely tied to the GenLayer stack, not a generic tool with GenLayer in the README.

For these projects, Contract quality is scored on the depth and correctness of the GenLayer integration (SDK, GenVM, consensus surfaces) instead of on a contract of its own. Say so in the note.

Not reasons to reject on their own

- A messy repository, rough docs, an unclear way to run it, a simple app, a common idea, a basic UI, or minor warnings. These lower scores. Together with weak GenLayer work they can support a quality reject, but the note has to state the concrete evidence, not an impression of effort.
- Deterministic helper contracts, when another contract in the repository provides meaningful nondeterministic consensus. Evaluate determinism across the whole repository, not per file.
- `run_nondet_unsafe`. It is legitimate SDK usage.
- Older SDK syntax or pinned dependencies.
- Contracts not found where you expected them. Search the complete repository, including the root and unconventional directories, before concluding a contract is missing.
- Reviewer infrastructure failures. A failed clone, a rate limit, or a broken local environment is our problem, not the project's. Retry or escalate.
- Private or temporarily inaccessible evidence. Escalate for parent review instead of rejecting.

Score floors and the gate

Fit 0 is a reject. Fit 1 is still eligible for a low-scoring accept.

Step 2: Score each part from 0 to 5

← Project Review Criteria

Does this project really need GenLayer, or could a normal app do the same thing?

Score	What it means
0	It does not use GenLayer. Reject.
1	A normal app could do the same. The AI is only for small or fun things.
2	It uses AI, but nothing important depends on it. A normal app would still be fine.
3	It uses AI for something real, and GenLayer helps. A normal app could still work too.
4	It decides something important, where a real result or real money is at stake, and no single party should decide it alone.
5	It cannot work without GenLayer. The result matters, and only GenLayer can produce it in a fair way.

Ask these questions:

- Is consensus resolving conflicting incentives, or is it only producing a better AI answer?
- What breaks if a centralized operator makes the decision instead?
- Does the outcome affect a payout, claim, eligibility decision, escrow, moderation action, game result, ranking, or another consequence?
- Does the use case justify GenLayer's additional cost and latency?
- If current external facts matter, does the contract fetch or verify them?
- Is this a specific, thoughtful application, or another generic dispute, grant, loan, moderation, or adjudication wrapper?

A game can score high if GenLayer is part of how the game really works, not just for fun messages.

A useful product is not automatically a strong GenLayer fit. Real usage and traction do not raise fit when the underlying trust model is weak. Note traction in the overall opinion instead.

2. Contract quality

Is the contract real and well made? Do the validators check the leader's real answer, or only the shape of it?

Score	What it means
0	

	No real contract, or the contract never uses the nondeterministic path.
1	An unmodified example, or format-only validation.
2	It works, but the validation is weak or does not fit the task.
3	A real contract. Validators verify the substantive result and the method fits the task.
4	Robust validation, failure paths handled, non-trivial logic and state.
5	Strong validation plus non-trivial architecture: more than one contract working together as a coherent system, or advanced code done well.

Rules that override the table:

- Validators must verify the leader's substantive result, not only format, labels, ranges, JSON validity, or generic plausibility. Format-only validation caps this category at 1.
- The Equivalence Principle choice has to fit the output type.
- Source-dependent decisions should verify evidence through web fetching, attestations, or another credible source path.
- 4 and 5 require both robust validation and non-trivial architecture. A single LLM call plus storage is not enough.
- Deterministic helper contracts contribute positively when they are part of a coherent multi-contract system.

3. Engineering

Is this real work, with judgment behind it?

Engineering is about structure, integration, and failure handling. It is not about volume. Do not score on commit count or file count.

Score	What it means
0	Empty, or it does not run.
1	No real structure. Integration copied without adaptation, no usable setup path.
2	It runs, but it is small and simple, with little structure and unclear setup.
3	Coherent structure. It builds and runs from documented steps. The contract and the app are wired together deliberately.
4	

	Clean structure, clear setup and configuration, failure states handled (network, wallet, contract errors), visible judgment in how it is put together.
5	Very strong. Clean and well organized, easy to run, good docs, deployment and edge cases considered.

Tests are a plus, not a requirement.

4. Frontend / UX

Is there an app that really connects to the contract?

Score	What it means
0	No app. This is fine for a contract-only project or for tooling.
1	A very basic page, or an app that does not claim contract integration and only touches peripheral reads.
2	A real but rough connection to the contract on the main flow.
3	A working app that covers the main steps, with transaction states visible.
4	A good deployed app that handles the full flow, including wallet, network, and understandable errors.
5	A very good app, smooth and complete.

On integration claims:

- Claimed but simulated integration is a reject for the whole project, not a low Frontend score.
- Real but flawed or incomplete integration is a low score, or request more info if it blocks verification.

The three outcomes

Accept

The gate passes and the scores are honest. Low scores can still be accepted. Fit 1 is eligible.

Request more info

Use it only for a narrow, fixable blocker. Typical cases:

- The contract code is not in the repository. A Studio or deployment link on its own is not enough to review, since the code has to be readable.
- Integration is real but something specific prevents verification.

How to use it:

- Finish the whole review first, then consolidate every mandatory correction into one request. Do not drip-feed requirements.
- Separate mandatory requirements from optional advice in the note.
- On resubmission, reject if the requested fix is still unresolved.
- If the requested fix is complete and the project is functional, accept. Do not introduce another pre-existing requirement that you could have raised the first time.
- Appeals challenge the original decision. New work belongs in a resubmission or a new submission.

Reject

Reject when the gate fails, or when the quality problems are concrete, documented, and add up. Whatever the outcome, the note has to be clear enough for the builder to act on. Reject or more info makes no sense if the feedback is unclear.

Reviewing several submissions from the same builder

Compare near-identical submissions as a cohort, and do not punish legitimate reuse.

- Score genuinely distinct projects normally.
- Score repeated scaffolds with little new depth conservatively.
- Quality reject branding, prompt, theme, or minor-code variants that do not create distinct value.

Keep accusations of reward farming out of user-facing feedback unless the evidence is conclusive. Raise the pattern internally instead.

Feedback standard

Feedback should be brief and natural, and contain:

- the outcome;
- the main reason;
- one concrete next step when applicable.

Do not mention internal points or category scores, do not sound robotic, do not summarize the builder's own product back to them, and do not use vague statements such as "needs more work".

Two quick examples

A small game where the AI only writes fun messages. A normal app could do this, so GenLayer fit is 1, even if the game is fun.

One small contract and one simple page, with a big name. If the validators only check the format of the answer, contract quality is capped at 1. A good name and a working

AI call are not enough for a middle score.

What to send back

For each of the four parts, a score from 0 to 5 and one short line on why. Note whether the project has a live app, a video, or a post. Add one line with your overall opinion, and the outcome you propose.